



Deriving Types From Types

typeof and keyof

Indexed Access Types & Mapped Types

Conditional Types

Built-in Utility Types

More!

Understanding Indexed Access Types

```
type AppUser = {  
  name: string;  
  age: number;  
  permissions: {  
    id: string;  
    title: string;  
    description: string;  
  }[];  
};  
  
type Perms = {  
  id: string;  
  title: string;  
  description: string;  
}[];
```

If you want to extract a part of an object type

into a separate type, you can do so

```
type AppUser = {  
  name: string;  
  age: number;  
  permissions: {  
    id: string;  
    title: string;  
    description: string;  
  }[];  
};
```

```
type Perms = AppUser;
```

by referring to that object type here,

```
type AppUser = {  
  name: string;  
  age: number;  
  permissions: {  
    id: string;  
    title: string;  
    description: string;  
  }[];  
};
```

```
type Perms = AppUser['permissions'];
```

Introducing Mapped Types

```
File Edit Selection View Go Run Terminal Help
← → Section 10 - Deriving Types From Types

mapped-types.ts U X
mapped-types.ts > mathOperations
1 type Operations = {
2   add: (a:number,b:number) => number;
3   subtract: (a:number,b:number) => number;
4 };
5
6
7 let mathOperations: Operations = {
8   add(a:number, b:number){
9     return a + b;
10  },
11
12   subtract(a:number, b:number){
13     return a-b;
14   }
15 };
16
17 //let's say I want to store the result of the 'mathOperations'
18 type Results = {
19   add: number,
20   subtract: number
21 };
22
23 let mathResults: Results = {
24   add: mathOperations.add(1,2),
25   subtract: mathOperations.subtract(5,2)
26 };
```

So far nothing new! But as you can see here, the "results" object is kind of related to this "operations" object type.

```
1 type Operations = {
2   add: (a: number, b: number) => number;
3   subtract: (a: number, b: number) => number;
4 };
5
6 type Results = {
7   add: number;
8   subtract: number;
9 };
10
11 let mathOperations: Operations = {
12   add(a: number, b: number) {
13     return a + b;
14   },
15   subtract(a: number, b: number) {
16     return a - b;
17   }
18 };
19
20 let mathResults: Results = {
21   add: mathOperations.add(1,2),
22   subtract: mathOperations.subtract(5,2)
23 };
```

It has the same property names here

```

6 type Results = {
7   add: number;
8   subtract: number;
9 };
10
11 let mathOperations: Operations = {
12   add(a: number, b: number) {
13     return a + b;
14   },
15   subtract(a: number, b: number) {
16     return a - b;
17   }
18 };

```

and it then just stores the results

of calling these methods.

```

5
6 type Results = {
7   add: number;
8   subtract: number;
9 };
10
11 let mathOperations: Operations = {
12   add(a: number, b: number) {
13     return a + b;
14   },
15   subtract(a: number, b: number) {
16     return a - b;
17   }
18 };

```

Of course here it's a very simple type,

so defining it like this is no problem.

```

6 type Results = {
7   add: number;
8   subtract: number;
9 };
10
11 let mathOperations: Operations = {
12   add(a: number, b: number) {
13     return a + b;
14   },
15   subtract(a: number, b: number) {
16     return a - b;
17   }
18 };

```

But if this would be a more complex object,

it could become annoying to define it like this

if it's closely related to another object.

And that's exactly where mapped-types can help you.

```

1  type Operations = {
2    add: (a: number, b: number) => number;
3    subtract: (a: number, b: number) => number;
4  };
5
6  type Results = {
7    add: number;
8    subtract: number;
9  };
10
11 let mathOperations: Operations = {
12   add(a: number, b: number) {
13     return a + b;
14   },
15   subtract(a: number, b: number) {

```

Because instead of manually defining

this object type like this,

```

type Operations = {
  add: (a: number, b: number) => number;
  subtract: (a: number, b: number) => number;
};

type Results<T> = {
  [Key in keyof T]: number;
};

let mathOperations: Operations = {
  add(a: number, b: number) {
    return a + b;
  },
  subtract(a: number, b: number) {
    return a - b;
  }
};

```

Well, and with that we created a mapped- type in the end.

Readonly Types & Optional Mapping

127 Readonly Types & Optional Mapping

```

1  type Operations = {
2    add: (a: number, b: number) => number;
3    subtract: (a: number, b: number) => number;

```

127. Readonly Types & Optional Mapping

```
1  type Operations = {
2    | add: (a: number, b: number) => number;
3    | subtract: (a: number, b: number) => number;
4  };
5
6  type Results<T> = {
7    | [Key in keyof T]: number
8  };
9
10 let mathOperations: Operations = {
11   | add(a: number, b: number) {
12     | return a + b;
13   },
14   | subtract(a: number, b: number) {
15     | return a - b; }
```

Specifically, they also allow you to, for example,

make properties optional,

even though if they're not optional

b in the original type you're mapping from or vice versa.

```
a: number, b: number) {
```

They allow you to make properties non-optional

if they have been optional before,

```
ber, b: number) {
```

and I'll start by showing you

```
umber, b: number) {
```

how to make properties optional.

```
act(a: number, b: number) {
```

Right, now we're simply mapping all the properties

```
urn a - b;
```



```
1 type Operations = {
2   add: (a: number, b: number) => number;
3   subtract: (a: number, b: number) => number;
4 };
5
6 type Results<T> = {
7   [Key in keyof T]: number
8 };
9
10 let mathOperations: Operations = {
11   add(a: number, b: number) {
12     return a + b;
13   },
14   subtract(a: number, b: number) {
15     return a - b;
16   },
17   of the incoming type, so of the operations type,
```

```
1 type Operations = {
2   add: (a: number, b: number) => number;
3   subtract: (a: number, b: number) => number;
4 };
5
6 type Results<T> = {
7   [Key in keyof T]: number
8 };
9
10 let mathOperations: Operations = {
11   add(a: number, b: number) {
12     return a + b;
13   },
14   subtract(a: number, b: number) {
15     return a - b;
16   },
17   and since none of these two properties, add and subtract,
```

are optional here, none of them are optional here either,

```
let mathResults: Results<Operations> = {
  add: mathOperations.add(1, 2),
};
```

```
let mathResults: Results<Operations> = {
  add: mathOperations.add(1, 2),
};
```

omitting subtract gives us an error,

because subtract is missing, but required.

Now, what if we want more flexibility in our results?

```
1 type Operations = {
2   add: (a: number, b: number) => number;
3   subtract: (a: number, b: number) => number;
4 };
5
6 type Results<T> = {
7   [Key in keyof T]?: number
8 };
9
10 let mathOperations: Operations = {
11   add(a: number, b: number) {
12     return a + b;
13   },
14   subtract(a: number, b: number) {
15     return a - b;
16   }
17 }
```

Well, you can simply add a question mark here

and that makes all these properties you're mapping

into this object type here optional.


```
let mathResults: Results<Operations> = {  
  subtract: mathOperations.subtract(5, 2),  
};
```

That is now allowed.

Now, the opposite would also work.

```
type Operations = {  
  add?: (a: number, b: number) => number;  
  subtract?: (a: number, b: number) => number;  
};  
  
type Results<T> = {  
  [Key in keyof T]: number  
};  
  
let mathOperations: Operations = {  
  add(a: number, b: number) {  
    return a + b;  
  },  
  subtract(a: number, b: number) {  
    return a - b;  
  },  
};
```

Let's say add and subtract were actually optional methods.

```
15     return a - b;  
16   },  
17 };  
18  
19 let mathResults: Results<Operations> = {  
20   add: mathOperations.add(1, 2),  
21   subtract: mathOperations.subtract(5, 2),  
22 };  
23
```

Now we get an error down here,

```

let mathOperations: Operations = {
  add(a: number, b: number) {
  },
  subtract(a: number, b: number) {
    return a - b;
  },
};

let mathResults: Results<Operations> = {
  add: mathOperations.add(1, 2),
  subtract: mathOperations.subtract(5, 2),
};

```

that potentially doesn't exist,

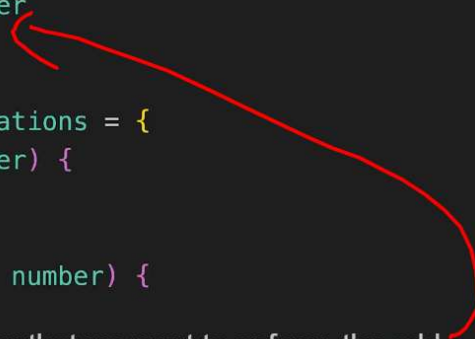
mapped-types.ts 2, U

```

1  type Operations = {
2    add?: (a: number, b: number) => number;
3    subtract?: (a: number, b: number) => number;
4  };
5
6  type Results<T> = {
7    [Key in keyof T]: number
8  };
9
10 let mathOperations: Operations = {
11   add(a: number, b: number) {
12     return a + b;
13   },
14   subtract(a: number, b: number) {
15     return a - b;
16   },

```

Let's instead say that we want to enforce the add



```

5
6 type Results<T> = {
7   [Key in keyof T]: number
8 };
9
10 let mathOperations: Operations = {
11   add(a: number, b: number) {
12     return a + b;
13   },
14   subtract(a: number, b: number) {
15     return a - b;
16   },

```

and subtract keys, for whatever reason, in this object.

```

1 type Operations = {
2   add?: (a: number, b: number) => number;
3   subtract?: (a: number, b: number) => number;
4 };
5
6 type Results<T> = {
7   [Key in keyof T]: number
8 };
9
10 let mathOperations: Operations = {
11   add(a: number, b: number) {
12     return a + b;
13   },
14   subtract(a: number, b: number) {
15     return a - b;

```

At the moment, since they're optional in operations,

```

type Results<T> = {
  [Key in keyof T]: number
};

let mathOperations: Operations = {
  add(a: number, b: number) {
    return a + b;
  },
  subtract(a: number, b: number) {
    return a - b;
  },

```

they would be optional here in results as well

(a: number, b: number) => number because we're mapping those properties

```
let mathResults: Results<Operations> = {}
```

Defining neither of these would be fine.

```
type Operations = {
  add?: (a: number, b: number) => number;
  subtract?: (a: number, b: number) => number;
};

type Results<T> = {
  [Key in keyof T]: number
};

let mathOperations: Operations = {
  add(a: number, b: number) {
    return a + b;
  },
  subtract(a: number, b: number) {
    return a - b;
  },
};
```

since add and subtract are optional up here.

```
type Results<T> = {
  [Key in keyof T]: number
};

let mathOperations: Operations = {
  add(a: number, b: number) {
    return a + b;
  },
  subtract(a: number, b: number) {
    return a - b;
  },
};
```

Now, if you wanna remove that optionality,

```

5
6 type Results<T> = {
7   [Key in keyof T]-?: number
8 };
9
10 let mathOperations: Operations = {
11   add(a: number, b: number) {
12     return a + b;
13   },
14   subtract(a: number, b: number) {
15     return a - b;
16   }

```

you can do so by adding minus question mark here.

```

type Results<T> = {
  [Key in keyof T]-?: number
};

let mathOperations: Operations = {
  add(a: number, b: number) {
    return a + b;
  },
  subtract(a: number, b: number) {
    return a - b;
  }
}

```

for removing the optional flag

```

5
6 type Results<T> = {
7   [Key in keyof T]-?: number
8 };
9
10 let mathOperations: Operations = {
11   add(a: number, b: number) {
12     return a + b;
13   },
14   subtract(a: number, b: number) {
15     return a - b;
16   }
}

```

when working with mapped types, as we're doing it here.

```

let mathResults: Results<Operations> = {
  // ...
};

```

you see I'm getting an error down here telling me

```

type Operations = {
  add: (a: number, b: number) => number;
  subtract: (a: number, b: number) => number;
};

type Results<T> = {
  [Key in keyof T]?: number;
};

let mathOperations: Operations = {
  add(a: number, b: number) {
    return a + b;
  },
  subtract(a: number, b: number) {
    return a - b;
  },
}

```

to be optional properties in the results type,

```

5 |
6 | type Results<T> = {
7 |   readonly [Key in keyof T]?: number;
8 | };
9 |
10 | let mathOperations: Operations = {
11 |   add(a: number, b: number) {
12 |     return a + b;
13 |   },
14 |   subtract(a: number, b: number) {
15 |     return a - b;
16 |   },

```

but if we want to make the results read only

: we can do so by adding the read only flag

Now, as a little side note,


```

1  type Operations = {
2    readonly add: (a: number, b: number) => number;
3    subtract: (a: number, b: number) => number;
4  };
5
6  type Results<T> = {
7    readonly [Key in keyof T]?: number;
8  };
9
10 let mathOperations: Operations = {
11   add(a: number, b: number) {
12     return a + b;
13   },
14   subtract(a: number, b: number) {
15     return a - b;

```

you can always add the read only flag

when defining an object type

```

type Operations = {
  readonly add: (a: number, b: number) => number;
  readonly subtract: (a: number, b: number) => number;
};

type Results<T> = {
  readonly [Key in keyof T]?: number;
};

let mathOperations: Operations = {
  add(a: number, b: number) {
    return a + b;
  },
  subtract(a: number, b: number) {
    return a - b;

```

to make a property or method read only

```
10 let mathOperations: Operations = {
17 };
18
19 let
20
21 (property) add?: any
22 };
23
24 mathResults.add = 10;
```

Cannot assign to 'add' because it is a read-only property. ts(2540)

[View Problem \(↵F8\)](#) No quick fixes available

because it's a read only property.

If we remove 'readonly'

```
type Results<T> = {
  [Key in keyof T]?: number;
};
```

```
24 mathResults.add = 10;
```

the error went away, so that really makes a difference.

so let's say we already had read only here

```

type Operations = {
  readonly add: (a: number, b: number) => number;
  readonly subtract: (a: number, b: number) => number;
};

```

```

type Results<T> = {
  readonly [Key in keyof T]?: number;
};

```

```

let mathOperations: Operations = {
  add(a: number, b: number) {
    return a + b;
  },
  subtract(a: number, b: number) {
    return a - b;
  }
};

```

on the add and subtract method.

`a - b;`
In that case, we would have read only properties

```

5  type Results<T> = {
6    [Key in keyof T]?: number;
7  };
8
9
10 let mathOperations: Operations = {
11   add(a: number, b: number) {
12     return a + b;
13   },
14   subtract(a: number, b: number) {
15     return a - b;
16   }
};

```

`number, b: number) {`
`return a - b;`
we always also take their flags

```

type Operations = {
  readonly add: (a: number, b: number) => number;
  readonly subtract: (a: number, b: number) => number;
};

```



```

type Results<T> = {
  [Key in keyof T]?: number;
};

```

```

let mathOperations: Operations = {
  add(a: number, b: number) {
    return a + b;
  },
  subtract(a: number, b: number) {
    return a - b;
  }
};

```

so the add and subtract properties in this type

a: number, b: number) => number; would be read only if they are read only

15 mapped-types.ts 1, 0

```

1 type Operations = {
2   readonly add: (a: number, b: number) => number;
3   readonly subtract: (a: number, b: number) => number;
4 };

```

```

6 type Results<T> = {
7   [Key in keyof T]?: number;
8 };

```

```

10 let mathOperations: Operations = {
11   add(a: number, b: number) {
12     return a + b;
13   },
14   subtract(a: number, b: number) {
15     return a - b;
16   },

```

in the original type, and therefore,

```

18
19 let mathResults: Results<Operations> = {
20   add: mathOperations.add(5, 1),
21   subtract: mathOperations.subtract(5, 2),
22 };
23
24 mathResults.add = 10;

```

we would get this error.

But now let's say we wanna remove

the read only configuration.

```

type Operations = {
  readonly add: (a: number, b: number) => number;
  readonly subtract: (a: number, b: number) => number;
};

type Results<T> = {
  [Key in keyof T]?: number;
};

let mathOperations: Operations = {
  add(a: number, b: number) {
    return a + b;
  },
  subtract(a: number, b: number) {
    return a - just as you could with the question mark,
  }
}

```

```

type Results<T> = {
  -readonly [Key in keyof T]?: number;
};

let mathOperations: Operations = {
  add(a: number, b: number) {
    return a + b;
  },
  subtract(a: number, b: number) {
    return a - b;
  }
};

```

you can add minus read only here,

Exploring Template Literal Types

```

// const greeting = `Hi there, ${mainUserName}.`;

type ReadPermissions = 'no-read' | 'read';
type WritePermissions = 'no-write' | 'write';

type FilePermissions =
  | 'no-read-write'
  | 'read-no-write'
  | 'no-read-no-write'
  | 'read-write';

```

I wanna have all these combinations in file permissions,

```

4  ⚡
5  type ReadPermissions = 'no-read' | 'read';
6  type WritePermissions = 'no-write' | 'write';
7
8  type FilePermissions =
9    | 'no-read-write'
10   | 'read-no-write'
11   | 'no-read-no-write'
12   | 'read-write';
13

```

combinations of all these string literal values,

of all the text values, if you wanna call them like this,


```

3  // const greeting = `Hi there, ${mainUserName}.`;
4
5  type ReadPermissions = 'no-read' | 'read';
6  type WritePermissions = 'no-write' | 'write';
7
8  type FilePermissions =
9      | 'no-read-write'
10     | 'read-no-write'
11     | 'no-read-no-write'
12     | 'read-write';
13

```

Well, of course, we could do it by hand like this,

but we could instead also use

this template literal type feature offered by TypeScript,

```

type ReadPermissions = 'no-read' | 'read';
type WritePermissions = 'no-write' | 'write';

type FilePermissions = `

```

and you do that by adding the backticks here again,

```

TS template-literal-types.ts U
1 // const mainUserName = 'Max';
2
3 // const greeting = `Hi there, ${mainUserName}.`;
4
5 type ReadPermissions = 'no-read' | 'read';
6 type WritePermissions = 'no-write' | 'write';
7
8 type FilePermissions = `${ReadPermissions}-${WritePermissions}`;
9

```

And this is now a valid type definition,

```

types template-literal-types.ts U
8 type FilePermissions = `${ReadPermissions}-${WritePermissions}`;
9
10 type DataFile = {
11   data: string;
12   permissions: FilePermissions;
13 };
14 type DataFileEventNames = "dataChanged" | "permissionsChanged";
15 type DataFileEvents = `${keyof DataFile}Changed`;
16
17 type DataFileEvents = {
18
19 };
20

```

```

0  type DataFile = {
1    data: string;
2    permissions: FilePermissions;
3  };
4
5  type DataFileEventNames = `${keyof DataFile}Changed`;
6  💡
7  type DataFileEvents = {
8    [Key in DataFileEventNames]: () => void;
9  };

```

Introducing Conditional Types

```

conditional.ts U ●
1  type StringArray = string[];
2  type ElementType = StringArray[number];
   type ElementType = string

```

But as a result, ElementType here would be string.

conditional.ts U ●

```
1 type StringArray = string[];  
2 type ElementType = StringArray[number];  
   type ElementType = string
```

But as a result, ElementType here would be string.

```
type StringArray = string[];  
type ElementType = StringArray[number];
```



Now let's say you want to build a generic utility type

that you can use on all kinds of arrays

to extract the information about the types

that is managed by those arrays.

```
1 type StringArray = string[];
2 type ElementType<T> = StringArray[number];
3
```

You could achieve this by making ElementType generic,

```
type StringArray = string[];
type ElementType<T> = T[number];
```

However this fails because TypeScript doesn't know

whether T will really be an array.

Of course you could convince TypeScript

by adding a constraint here,

conditional.ts 2, U ●

```
1 type StringArray = string[];  
2 type ElementType<T extends > = T[number];  
3
```

telling it that T will always extend some array type.

s conditional.ts U ●

```
1 type StringArray = string[];  
2 type ElementType<T extends any[]> = T[number];  
3
```

So an array full of anything,

where we don't care about the actual type


```

1 type StringArray = string[];
2 type ElementType<T extends any[]> = T[number];
3 type Example1 = string
4 type Example1 = ElementType<StringArray>

```

So now you would have a helper ElementType

which can be used on any array type

to extract information about the type

of value stored in the array,

But one potential problem in a more complex applications

```

TS conditional.ts U
1 type StringArray = string[];
2 type ElementType<T extends any[]> = T[number];
3
4 type Example1 = ElementType<StringArray>
5

```

is that you can't use ElementType on anything

that's not an array.

```
type StringArray = string[];
type ElementType<T extends any[]> = T[number];

type Example1 = ElementType<StringArray>
💡
let text = 'hello';|

type Example2 = ElementType<typeof text>;
```

that text variable, I would get an error

```
conditional.ts 1, U •
1 type StringArray = string[];
2 type ElementType<T extends any[]> = T[number];
3
4 type Example1 = ElementType<StringArray>
5
6 let text = 'hello';
7
8 type Example2 = ElementType<typeof text>;
```

Type 'string' does not satisfy the constraint 'any[]'. ts(2344)

[View Problem \(\F8\)](#) No quick fixes available

that my string here is not an array.

And of course the same would be true

```

conditional.ts 1, U
1  type StringArray = string[];
2  type ElementType<T extends any[]> = T[number];
3
4  type Example1 = ElementType<StringArray>
5  ⚡
6  let text = 1;
7
8  type Example2 = ElementType<typeof text>;

```

if this were a number.

I would get an error.

And this of course is not necessarily a problem.

```

es TS conditional.ts 1, U
1  type StringArray = string[];
2  type ElementType<T extends any[]> = T[number];
3
4  type Example1 = ElementType<StringArray>
5  ⚡
6  let text = 1;
7
8  type Example2 = ElementType<typeof text>;

```

I mean we built a utility type ElementType here,

which can only be used on arrays,

that's not necessarily bad or a problem.

But you could build an even more generic helper type

which allows you to extract information from other types

by using this conditional type feature I want to show you.

```
type StringArray = string[];  
// type ElementType<T extends any[]> = T[number];  
  
// type Example1 = ElementType<StringArray>  
  
// let text = 1;  
  
// type Example2 = ElementType<typeof text>;
```

So, for that I'll actually comment out this code,

```
1 type StringArray = string[];  
2 // type ElementType<T extends any[]> = T[number];  
3  
4 // type Example1 = ElementType<StringArray>  
5  
6 // let text = 1;  
7  
8 // type Example2 = ElementType<typeof text>;  
9  
10 type GetElementType
```



And now here I'll again make it generic,

but I won't add that constraint

that it only works on arrays.

```
10 type GetElementType<T> = |
```

Instead, we can make this a conditional type

by adding a ternary expression-like syntax

conditional.ts 1, U ●

```
1 type StringArray = string[];
2 // type ElementType<T extends any[]> = T[number];
3
4 // type Example1 = ElementType<StringArray>
5
6 // let text = 1;
7
8 // type Example2 = ElementType<typeof text>;
9
10 type GetElementType<T> = T extends any[]
```

and check whether it extends, any array.

```
10 type GetElementType<T> = T extends any[]
```

Now here I'm not defining a constraint

because I'm not using this here

```
9
10 type GetElementType<T> = T extends any[]
```

between the angle brackets,

conditional.ts 1, U

```
type StringArray = string[];
// type ElementType<T extends any[]> = T[number];

// type Example1 = ElementType<StringArray>

// let text = 1;

// type Example2 = ElementType<typeof text>;
type GetElementType<T> = T extends any[]
```

instead this is the syntax for writing a condition

in TypeScript, it is kind of writing equals, you could say,

TS conditional.ts 3, U

```
1 type StringArray = string[];
2 // type ElementType<T extends any[]> = T[number];
3
4 // type Example1 = ElementType<StringArray>
5
6 // let text = 1;
7
8 // type Example2 = ElementType<typeof text>;
9
10 type GetElementType<T> = T extends any[]
```

in TypeScript, it is kind of writing equals, you could say,

```
type GetElementType<T> = T extends any[]
```

I'm checking whether the type I'm getting

```
9  
10 type GetElementType<T> = T extends any[]
```

is an array full of anything.

```
8 // type Example2 = ElementType<typeof text>;  
9  
10 type GetElementType<T> = T extends any[] ?
```

Then you add a question mark and then

after the question mark, you define the type

that should be returned, you could say,

```
8 // type Example2 = ElementType<typeof text>;  
9   
10 type GetElementType<T> = T extends any[] ?
```

by GetElementType if this condition is met.

So if the type we're using GetElementType on

is some array.

TS conditional.ts 1, U ●

```
1 type StringArray = string[];
2 // type ElementType<T extends any[]> = T[number];
3
4 // type Example1 = ElementType<StringArray>
5
6 // let text = 1;
7
8 // type Example2 = ElementType<typeof text>;
9
10 type GetElementType<T> = T extends any[] ? T[number]
```

And in my case here, that will be this type here

```
7
8 // type Example2 = ElementType<typeof text>;
9
10 type GetElementType<T> = T extends any[] ? T[number]
```

where I use this special syntax for extracting the type

```
// let text = 1;

// type Example2 = ElementType<typeof text>;

type GetElementType<T> = T extends any[] ? T[number]
```

of the elements in that array.

```

8 // type Example2 = ElementType<typeof text>;
9
10 type GetElementType<T> = T extends any[] ? T[number]

```

And I know that I'm working

with an array here due to this condition.

Now just as in ternary expressions in JavaScript,

you then also must add a colon to define an else case.

```

9
0 type GetElementType<T> = T extends any[] ? T[number] :

```

What should be the type returned by GetElementType

```

7
8 // type Example2 = ElementType<typeof text>;
9
10 type GetElementType<T> = T extends any[] ? T[number] :

```

if this condition here is not met?

And that can be anything of your choice in the end.

```

type GetElementType<T> = T extends any[] ? T[number] : T;

```

For example, it could be T, so the unchanged type,

```

7
8 // type Example2 = ElementType<typeof text>;
9
10 type ElementType<T> = T extends any[] ? T[number] : never;

```

it could be never, to kind of make it clear

```

8 // type Example2 = ElementType<typeof text>;
9
10 type ElementType<T> = T extends any[] ? T[number] : never;

```

that this will not be a value

```

7
8 // type Example2 = ElementType<typeof text>;
9
10 type ElementType<T> = T extends any[] ? T[number] : never;

```

that should be returned by this type,

```

6 // let text = 1;
7
8 // type Example2 = ElementType<typeof text>;
9
10 type ElementType<T> = T extends any[] ? T[number] : never;
11

```

And this now, allows me to use

```

9
10 type ElementType<T> = T extends any[] ? T[number] : never;
11 type Example1 = ElementType<StringArray>;

```

```

conditional.ts U
1  type StringArray = string[];
2  // type ElementType<T extends any[]> = T[number];
3
4  // type Example1 = ElementType<StringArray>
5
6  // let text = 1;
7
8  // type Example2 = ElementType<typeof text>;
9
10 type type Example1 = string extends any[] ? T[number] : never;
11 type Example1 = ElementType<StringArray>;

```

And in that case example one will be string.

```

3
4  // type Example1 = ElementType<StringArray>
5
6  let text = 1;
7
8  // type Example2 = ElementType<typeof text>;
9
10 type ElementType<T> = T extends any[] ? T[number] : never;
11 type Example1 = ElementType<StringArray>;
12 type Example2 = ElementType<typeof text>;

```

```

s conditional.ts U
1  type StringArray = string[];
2  // type ElementType<T extends any[]> = T[number];
3
4  // type Example1 = ElementType<StringArray>
5
6  let text = 1;
7
8  // type Example2 = ElementType<typeof text>;
9
10 type type Example2 = never extends any[] ? T[number] : nev
11 type type Example2 = never extends any[] ? T[number] : nev
12 type Example2 = ElementType<typeof text>;

```

And example two will be never.

```

conditional.ts U
1  type StringArray = string[];
2  // type ElementType<T extends any[]> = T[number];
3
4  // type Example1 = ElementType<StringArray>
5
6  let text = 1;
7
8  // type Example2 = ElementType<typeof text>;
9  💡
10 type GetElementType<T> = T extends any[] ? T[number] : T;
11 type Example1 = GetElementType<StringArray>;
12 type Example2 = GetElementType<typeof text>;

```

Now it would be number if it would just return T here

```

conditional.ts U
1  type StringArray = string[];
2  // type ElementType<T extends any[]> = T[number];
3
4  // type Example1 = ElementType<StringArray>
5
6  let text = 1;
7
8  // type Example2 = ElementType<typeof text>;
9
10 type GetElementType<T> = T extends any[] ? T[number] : T;
11 type Example2 = number;
12 type Example2 = GetElementType<typeof text>;

```

In that case example two would be number,

```

1  type StringArray = string[];
2  // type ElementType<T extends any[]> = T[number];
3
4  // type Example1 = ElementType<StringArray>
5
6  let text = 1;
7
8  // type Example2 = ElementType<typeof text>;
9
10 type GetElementType<T> = T extends any[] ? T[number] : never;
11 type Example1 = GetElementType<StringArray>;
12 type Example2 = GetElementType<typeof text>;

```


Well, obviously these are advanced and to be honest,

niche features which you might not need all the time

and in very many projects.

But when building libraries or frameworks,

anything like this with TypeScript,

```
conditional.ts U X
1  type StringArray = string[];
2  // type ElementType<T extends any[]> = T[number];
3
4  // type Example1 = ElementType<StringArray>
5
6  let text = 1;
7
8  // type Example2 = ElementType<typeof text>;
9
10 type GetElementType<T> = T extends any[] ? T[number] : never;
11 type Example1 = GetElementType<StringArray>;
12 type Example2 = GetElementType<typeof text>;
```

you sometimes might need features like this

to be able to work with the different kinds

of values your library might need to deal with

when being used by our developers.

Conditional Types: Another Example

```
12 type Example2 = GetElementType<typeof text>;
13
14 function getFullName<T extends object>(person: T) {
15     if (
16         'firstName' in person &&
```

```

11 type Example2 = GetElementType<typeof text>;
12
13
14 function getFullName<T extends object>(person: T) {
15     if (
16         'firstName' in person &&
17         'lastName' in person &&
18         person.firstName &&
19         person.lastName
20     ) {
21         return `${person.firstName} ${person.lastName}`;
22     }
23
24     throw new Error('No first name and / or last name found.')
25 }

```

Now, that could be a function your application needs.

39 Another Example

```

11 type Example2 = GetElementType<typeof text>;
12 type Exam
13     function getFullName<T extends object>(person: T): string;
14 function getFullName<T extends object>(person: T) {
15     if (
16         'firstName' in person &&
17         'lastName' in person &&
18         person.firstName &&
19         person.lastName
20     ) {
21         return `${person.firstName} ${person.lastName}`;
22     }
23
24     throw new Error('No first name and / or last name found.')
25 }

```

Now, getFullName will return a string here.

That's what TypeScript infers.

But actually, that's not entirely correct.

```

    }
    throw new Error('No first name and / or last name found.')
}

```

It will actually return nothing. It will return never,

```

throw new Error('No first name and / or last name found.')

```

so to say, if we don't make it into this if check

Another Example

```
11 type Example1 = GetElementType<string> <T> <T>;
12 type Example2 = GetElementType<typeof text>;
13
14 function getFullName<T extends object>(person: T) {
15     if (
16         'firstName' in person &&
17         'lastName' in person &&
18         person.firstName &&
19         person.lastName
20     ) {
21         return `${person.firstName} ${person.lastName}`;
22     }
23     throw new Error('No first name and / or last name found.')
24 }
25
```

That's again a scenario where a conditional type

can help even though we're not building a utility type here.

'No first name and / or last name found.'

But this is one example

```
12 type Example2 = GetElementType<typeof text>;
13
14 function getFullName<T extends object>(person: T) {
15     if (
16         'firstName' in person &&
17         'lastName' in person &&
18         person.firstName &&
19         person.lastName
20     ) {
21         return `${person.firstName} ${person.lastName}`;
22     }
23     throw new Error('No first name and / or last name found.')
24 }
25
```

```

11 type Example1 = GetElementType<StringArray>;
12 type Example2 = GetElementType<typeof text>;
13
14 type FullnamePerson = { firstName: string; lastName: string };
15
16 function getFullname<T extends object>(  

17     person: T  

18 ): T extends FullnamePerson ? string : never {  

19     if (  

20         'firstName' in person &&  

21         'lastName' in person &&  

22         person.firstName &&  

23         person.lastName  

24     ) {  

25         return `${person.firstName} ${person.lastName}`;  

26     }  

27     throw new Error('No first name and / or last name found.');
```

or never anything depending on whether the object we get

```

type Example2 = GetElementType<typeof text>;

type FullnamePerson = { firstName: string; lastName: string };
type FullnameOrNothing<T> = T extends FullnamePerson ? string : never

function getFullname<T extends object>(  

    person: T  

): FullnameOrNothing<T> {  

    if (  

        'firstName' in person &&  

        'lastName' in person &&  

        person.firstName &&  

        person.lastName  

    ) {  

        return `${person.firstName} ${person.lastName}` as FullnameOrNothing<T>;  

    }  

    throw new Error('No first name and / or last name found.');
```

```

16
17 function getFullname<T extends object>(person: T): FullnameOrNothing<T> {  

18     if (  

19         'firstName' in person &&  

20         'lastName' in person &&  

21         person.firstName &&  

22         person.lastName  

23     ) {  

24         return `${person.firstName} ${person.lastName}` as FullnameOrNothing<T>;  

25     }  

26     throw new Error('No first name and / or last name found.');
```

```

16
17 function getFullName<T extends object>(person: T): FullnameOrNothing<T> {
18     if (
19         'firstName' in person &&
20         'lastName' in person &&
21         person.firstName &&
22         person.lastName
23     ) {
24         return `${person.firstName} ${person.lastName}` as FullnameOrNothing<T>;
25     }
26
27     throw new Error('No first name and / or last name found.');
```

```

28 }
29
30 const name1 = getFullName({});
31 const name2 = getFullName({firstName: 'Max', lastName: 'Schwarz Müller'});

```

Making Sense of the "infer" Keyword

```

1 function add(a: number, b: number) {
2     return a + b;
3 }
4
5 type AddFn = typeof add;
6 type ReturnValueType<T> = T extends (...args: any[]) =>
```

any amount of parameters which are of any type.

Utility Types

<https://www.typescriptlang.org/docs/handbook/utility-types.html>